MAT 586
Lab 6

Nolan R. H. Gagnon

Ex. 12.14

The figure below shows the distributions for the average degree of vaccinated nodes in four different scenarios. The black histogram was produced using a random network with 30% of its nodes randomly vaccinated. The gray histogram was also produced using a random network, but in this case 30% of its nodes were vaccinated using the "vaccinate a neighbor" strategy. From the histogram, it is clear that the average degree of vaccinated nodes increased under the "vaccinate a neighbor" strategy.

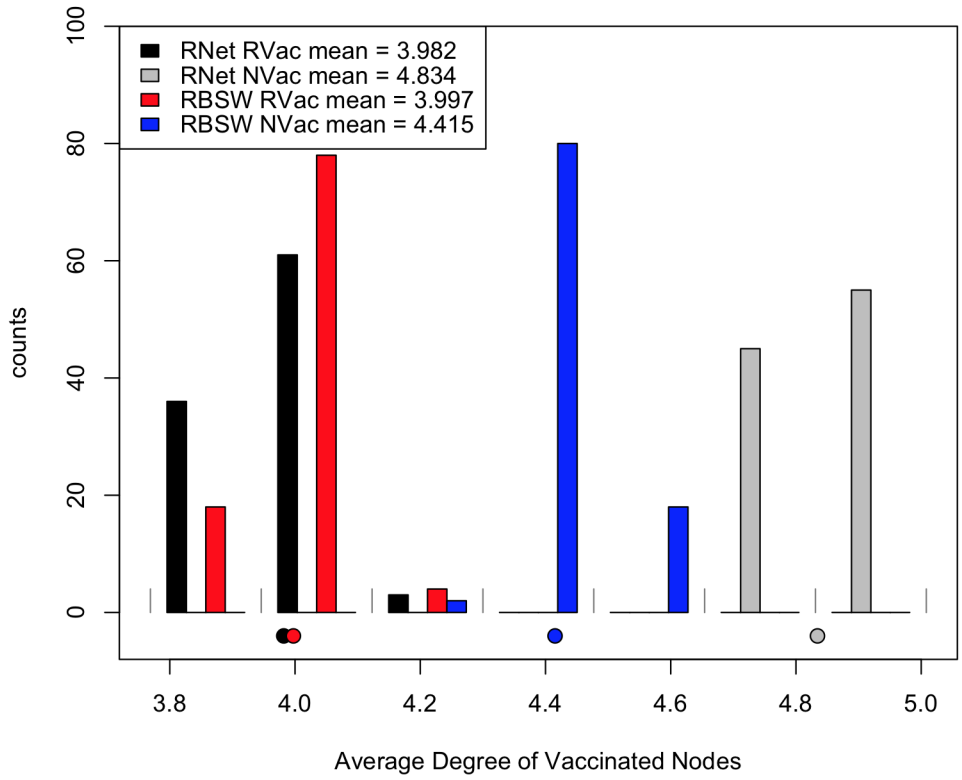

Figure 1: Histograms for Combinations of Network Types and Vaccination Strategies

The red histogram was produced using a Ring-Based Small World network (RBSW), with 30% of its nodes randomly vaccinated. The blue histogram was produced using an RSBW network, with 30% of its nodes vaccinated

using the "vaccinate a neighbor" strategy. For the RBSW networks, the "vaccinate a neighbor" strategy also increased the average degree of vaccinated nodes.

This pattern of having a higher average degree of vaccinated nodes under the "vaccinate a neighbor" strategy appeared in every simulation run. This makes sense, since, if a node is a neighbor, it has at least one connection already.

Two versions of the R script used to produce the figure on the previous page are provided below. One version is vectorized, the other is not.

exercise_ch12_no14.R

```
source("genRandomNet.R") # function to generate random networks
source("genRBSWNet.R") # function to generate random RBSW networks
source("genVacc.R") # function to perform random vaccination
source("genVaccNbor.R") # function to perform neighbor vaccination
source("multiHist.R") # function to plot multiple histograms in one figure

start <- proc.time()
## Generate 100 random networks with 1225 nodes and average degree 4
rand_nets.l <- vector("list", length = 100)
for (i in 1 : 100) {
   rand_nets.l[[i]] <- genRandomNet(n = 1225, k = 4)
}

## Generate 100 RBSW networks with radius 2 and rewiring probability 0.3
rbsw_nets.l <- vector("list", length = 100)
for (i in 1 : 100) {
   rbsw_nets.l[[i]] <- genRBSWNet(n = 1225, r = 2, p = 0.3)
}

## Allocate space for average degree vectors
data_rand_rand.v <- rep(NA, 100)
data_rand_nbor.v <- rep(NA, 100)
data_rbsw_rand.v <- rep(NA, 100)
data_rbsw_nbor.v <- rep(NA, 100)

## Find the average degree for each of the 200 networks
## generated above, under random vaccination and "vaccinate
## a neighbor" strategies
for (i in 1 : 100) {
   ## Get data for random nets, randomly vaccinated
```

```
  tmp.l <- genVacc(rand_nets.l[[i]], 0.3)
  vaccID.v <- tmp.l[[1]]
  data_rand_rand.v[i] <- mean(colSums(rand_nets.l[[i]][, vaccID.v, drop = FALSE]))

  ## Get data for RBSWs, randomly vaccinated
  tmp.l <- genVacc(rbsw_nets.l[[i]], 0.3)
  vaccID.v <- tmp.l[[1]]
  data_rbsw_rand.v[i] <- mean(colSums(rbsw_nets.l[[i]][, vaccID.v, drop = FALSE]))

  ## Get data for random nets, neighbor vaccination
  tmp.l <- genVaccNbor(rand_nets.l[[i]], 0.3)
  vaccID.v <- tmp.l[[1]]
  data_rand_nbor.v[i] <- mean(colSums(rand_nets.l[[i]][, vaccID.v, drop = FALSE]))

  ## Get data from RBSWs, neighbor vaccination
  tmp.l <- genVaccNbor(rbsw_nets.l[[i]], 0.3)
  vaccID.v <- tmp.l[[1]]
  data_rbsw_nbor.v[i] <- mean(colSums(rbsw_nets.l[[i]][, vaccID.v, drop = FALSE]))
}

## Plot histograms of the average degree data
data.l <- list(data_rand_rand.v, data_rand_nbor.v, data_rbsw_rand.v, data_rbsw_nbor.v)
multiHist(data.l, names.v = c(sprintf("RNet RVac mean = %.3f", mean(data_rand_rand.v)),
              sprintf("RNet NVac mean = %.3f", mean(data_rand_nbor.v)),
              sprintf("RBSW RVac mean = %.3f", mean(data_rbsw_rand.v)),
              sprintf("RBSW NVac mean = %.3f", mean(data_rbsw_nbor.v))),
    breaks = 7, xlab = 'Average Degree of Vaccinated Nodes')
print(proc.time() - start)
```

exercise_ch12_no14_vectorized.R

```
source("genRandomNet.R") # function to generate random networks
source("genRBSWNet.R") # function to generate random RBSW networks
source("genVacc.R") # function to perform random vaccination
source("genVaccNbor.R") # function to perform neighbor vaccination
source("multiHist.R") # function to plot multiple histograms in one figure

## Calculate the average degree of a vaccinated node
get_avg_degree <- function(A.m, vaccIDs.v)
```

```
{
   return(mean(colSums(A.m[, vaccIDs.v, drop = FALSE])))
}

## Get average degree data for a list of networks using a
## certain vaccination strategy
get_vacc_data <- function(nets.l, vacc_strat_f, prop)
{
   ## Get IDs of vaccinated nodes (for every network in the list)
   vaccIDs.l <- lapply(nets.l, FUN = function(A.m, p) vacc_strat_f(A.m, p = prop))

   ## Convert the element of vaccIDs.l from lists to vectors
   vaccIDs.l <- lapply(vaccIDs.l, FUN = function(x.l) unlist(x.l, use.names = FALSE))

   ## Return average degree data for the entire list of networks
   return(mapply(FUN = function(x, y) get_avg_degree(x, y), nets.l, vaccIDs.l))
}

start <- proc.time()

## Generate 100 random networks with 1225 nodes and average degree 4
rand_nets.l <- rep(list(genRandomNet(n = 1225, k = 4)), 100) # vectorized version

## Generate 100 RBSW networks with radius 2 and rewiring probability 0.3
rbsw_nets.l <- rep(list(genRBSWNet(n = 1225, r = 2, p = 0.3)), 100) # vectorized version



## Get average degree data
data_rand_rand.v <- get_vacc_data(rand_nets.l, genVacc, 0.3)
data_rand_nbor.v <- get_vacc_data(rand_nets.l, genVaccNbor, 0.3)
data_rbsw_rand.v <- get_vacc_data(rbsw_nets.l, genVacc, 0.3)
data_rbsw_nbor.v <- get_vacc_data(rbsw_nets.l, genVaccNbor, 0.3)

data.l <- list(data_rand_rand.v, data_rand_nbor.v, data_rbsw_rand.v, data_rbsw_nbor.v)

## Plot histograms of the average degree data
multiHist(data.l, names.v = c(sprintf("RNet RVac mean = %.3f", mean(data_rand_rand.v)),
              sprintf("RNet NVac mean = %.3f", mean(data_rand_nbor.v)),
              sprintf("RBSW RVac mean = %.3f", mean(data_rbsw_rand.v)),
              sprintf("RBSW NVac mean = %.3f", mean(data_rbsw_nbor.v))),
```

```
    breaks = 7, xlab = 'Average Degree of Vaccinated Nodes')
print(proc.time() - start)
```

The screen-shots below show the timing data for the two versions of the script.

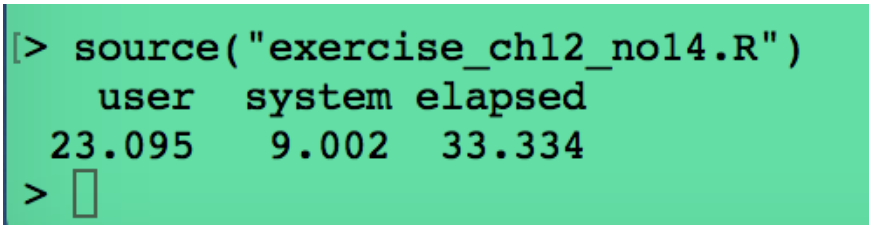

Figure 2: Non-vectorized R Script Timing Data

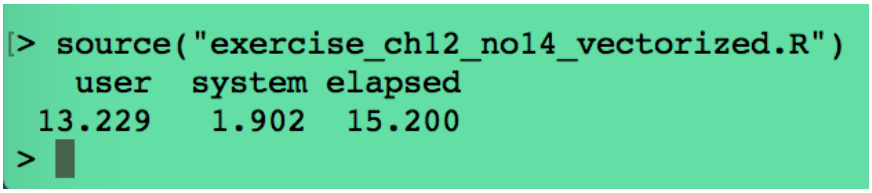

Figure 3: Vectorized R Script Timing Data

Clearly, the vectorized script is significantly faster.

■

Ex. 12.15

The figure below shows four histograms, each one showing the equilibrium infection level distribution after running the SIS-V simulation under a different set of conditions. The following list describes the different simulation conditions:

- **Black:** Random networks with 30% of the nodes randomly vaccinated
- **Gray:** Random networks with 30% of the nodes vaccinated using the "vaccinate a neighbor" strategy
- **Red:** RBSW networks with 30% of the nodes randomly vaccinated
- **Blue:** RBSW networks with 30% of the nodes vaccinated using the "vaccinate a neighbor" strategy

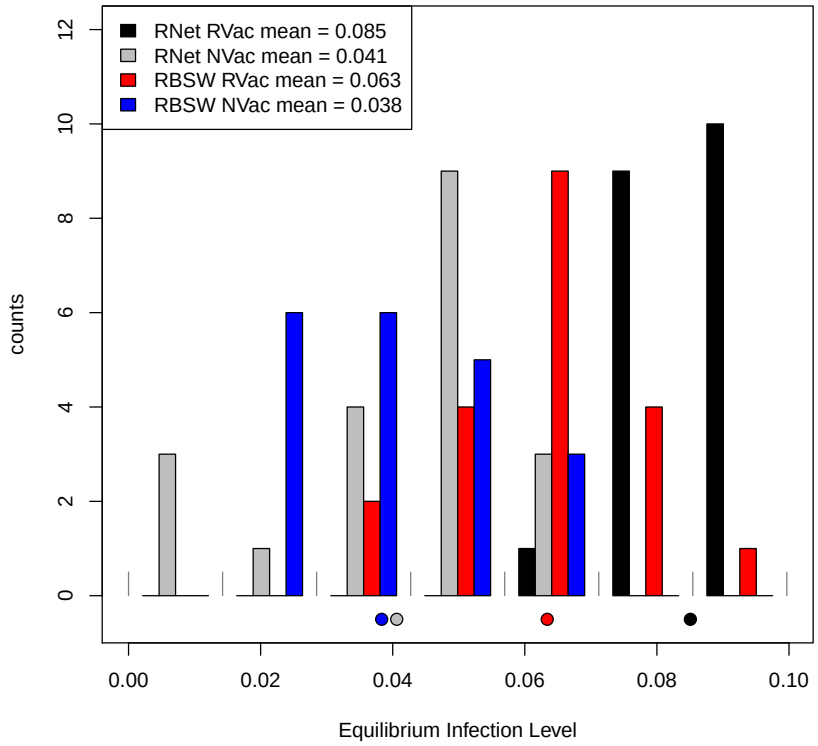

Figure 4: Histogram of Equilibrium Infection Levels for Different Networks and Vaccination Strategies

For both network types, the "vaccinate a neighbor" strategy results in a lower equilibrium infection level (on average). This is most likely due to the fact that "neighbors" are better connected, which means vaccinating them is going to slow down the spread of an infection.

The R script used to generated the above histogram is provided below. Note that in spite of the code being vectorized, it still runs quite slowly, due to the fact that it has to run 80 SIS-V simulations.

exercise_ch12_no15.R

```r
source("genRandomNet.R") # function to generate random networks
source("genRBSWNet.R") # function to generate random RBSW networks
source("genVaccNbor.R") # function to perform neighbor vaccination
source("genVacc.R") # function to perform random vaccination
source("multiHist.R") # function to plot multiple histograms in one figure
source("DNetSISVAID.R") # function to run SIS-V simulations

## Run SIS-V simulations on all networks in nets.l using
## vaccination strategy defined by vacc_strat_f;
## then return equilibrium infection levels for
## all networks.
get_SISV_equil_data <- function(nets.l, vacc_strat_f, prop)
{
   ## Get IDs of vaccinated nodes (for every network in the list)
   vaccIDs.l <- lapply(nets.l, FUN = function(A.m, p) vacc_strat_f(A.m, p = prop))

   ## Convert the elements of vaccIDs.l from lists to vectors
   vaccIDs.l <- lapply(vaccIDs.l, FUN = function(x.l) unlist(x.l, use.names = FALSE))

   ## Return simulation data for entire list of networks
   finalIMeans.v <- mapply(FUN = function(x, y) DNetSISVAID(x, vaccinatedID.v = y, phi = 5, numSteps =
       200, finalAvgLen = 30, displayEvery = 0), nets.l, vaccIDs.l)
   return(finalIMeans.v)
}

## Generate 20 random networks and 20 random RBSW networks.
rand_nets.l <- rep(list(genRandomNet(n = 1225, k = 4)), 20)
rbsw_nets.l <- rep(list(genRBSWNet(n = 1225, r = 2, p = 0.3)), 20)

## Get equilibrium infection level data for all network/vacc. strategy combinations
data_rand_rand.v <- get_SISV_equil_data(rand_nets.l, genVacc, 0.3)
data_rand_nbor.v <- get_SISV_equil_data(rand_nets.l, genVaccNbor, 0.3)
data_rbsw_rand.v <- get_SISV_equil_data(rbsw_nets.l, genVacc, 0.3)
data_rbsw_nbor.v <- get_SISV_equil_data(rbsw_nets.l, genVaccNbor, 0.3)

data.l <- list(data_rand_rand.v, data_rand_nbor.v, data_rbsw_rand.v, data_rbsw_nbor.v)

## Plot histograms of the average degree data
multiHist(data.l, names.v = c(sprintf("RNet RVac mean = %.3f", mean(data_rand_rand.v)),
```

```
                    sprintf("RNet NVac mean = %.3f", mean(data_rand_nbor.v)),
                    sprintf("RBSW RVac mean = %.3f", mean(data_rbsw_rand.v)),
                    sprintf("RBSW NVac mean = %.3f", mean(data_rbsw_nbor.v))),
     breaks = 7, xlab = 'Equilibrium Infection Level')
```

■